Mihai-Albert.com

Drilling Down to a Comfortable Level of Understanding

Out-of-memory (OOM) in Kubernetes – Part 2: The OOM killer and application runtime implications

February 13, 2022February 13, 2022 Mihai Albert

This is part 2 of a four-part article that looks into what happens in detail when Kubernetes runs into out-of-memory (OOM) situations and how it responds to them. You can find the article's full table of contents here (https://mihai-albert.com/2022/02/13/out-of-memory-oom-in-kubernetes-part-1-intro-and-topics-discussed/#table-of-contents) as part of the first post (https://mihai-albert.com/2022/02/13/out-of-memory-oom-in-kubernetes-part-1-intro-and-topics-discussed/).

Overcommit

We need to discuss overcommit, otherwise some of the stuff that comes next won't make sense, particularly if – like me – you're coming from the Windows world. What does overcommit mean? Simply put, the OS hands out more memory to processes than what it can safely guarantee.

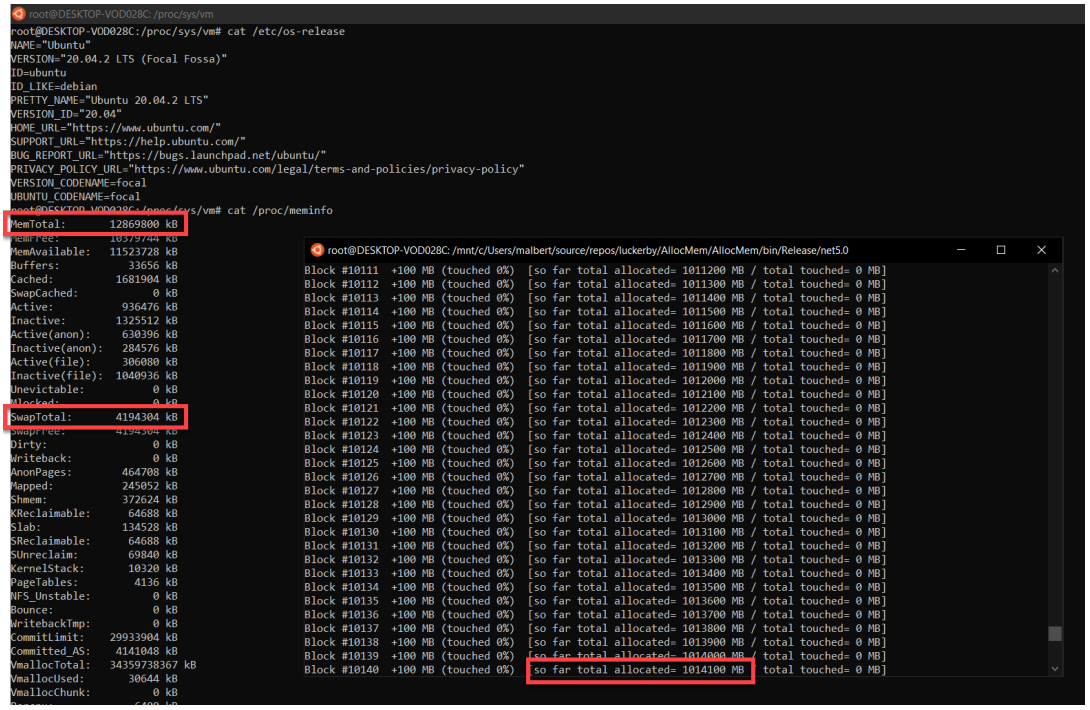
An example: we have a Linux system running with 12 GB RAM and 4 GB of swap. The system can be bare-metal, a VM, a WSL distribution running on Windows, etc as it doesn't matter from the standpoint of how overcommit works. Assume the OS uses 1 GB for its kernel and various components it runs. Process A comes along and requests to allocate 8 GB. The OS (Operating System) happily confirms, and process A gets a region of 8 GB inside its virtual address space that it can use. Next process B shows up and asks for 9 GB of memory. With overcommit turned on – which is usually the default setting – the OS is happy to process this request successfully as well, so now process B has a region of 7 GB inside its own virtual address space. Of course, if we sum 1+8+9 this shoots over the total quantity of memory that the OS knows it has at its disposal (12+4), but as long as processes A and B don't need to use all their memory *at once*, all is well.

And this works just fine as Linux – regardless of whether overcommit is used or not – won't rush to create memory pages for all that allocated memory, thinking "what if that memory I'm being requested won't ever actually be *used*?". In other words, it's being lazy (just as Windows is, as we discussed in depth here https://mihai-albert.com/2019/04/21/out-of-memory-exception/#your-reservation-is-now-confirmed (https://mihai-albert.com/2019/04/21/out-of-memory-exception/#your-reservation-is-now-confirmed)). But things change as both processes A and B *really* need all that memory they allocated, in the sense that they

start accessing or writing data to their respective memory locations. The OS cannot possibly satisfy both processes' requests to access all of their memory, as the OS simply doesn't have enough memory (be it RAM or swap) to *back* those locations. And the whole there's-plenty-of-memory-to-go-around-for-everyone charade comes to a stop when the real limit (12+2) is breached.

An analogy for overcommitting is a car insurance company that promises its clients that they'll cover the expenses if any of their clients' cars needs to be fixed. This works great as long as not everyone wants their car fixed at the same time, as that will put the insurance company out of business (and why you get those fine prints about pay exemptions in case of war, for example). Another parallel would be a bank whose clients all rush to redraw their deposits at once – no problem with this operation from the clients' perspective, as they're legally taking what's theirs, but the bank hasn't really planned for this scenario so most likely goes bust (known as a bank run).

To illustrate overcommit in action, here's a Linux system with ~12 GB RAM and 4 GB of swap configured and set to always overcommit that happily allocated 1 TB and keeps going. The 2 values highlighted on the left are the quantity of RAM and swap respectively, while the one to the bottom right is how much the tool did allocate successfully so far:



(https://luckerbyhome.files.wordpress.com/2021/11/linux_overcommit_1tb.png)

Figure 1 – Overcommit in action

There are 3 values that control the behavior for overcommit, and they're explained here https://www.kernel.org/doc/Documentation/vm/overcommit-accounting. (https://www.kernel.org/doc/Documentation/vm/overcommit-accounting.) The overcommit "mode" can be changed on the fly, without having to reboot the system, by using `sysctl vm.overcommit_memory=<value>`. The values:

- `0`: overcommit is enabled but Linux tunes it as it sees fit
- `1`: always overcommit in the sense that any memory allocation request is satisfied. The system seen above allocating 1 TB was configured like this
- `2`: don't overcommit. There's a given commit limit above which the OS will not go

If you've worked with Windows long enough, you'll recognize that "mode" 2 above is Linux behaving similarly. We've gone over the commit limit in Windows here (https://mihai-albert.com/2019/04/21/out-of-memory-exception/#your-reservation-is-now-confirmed). If you've read carefully the kernel documentation here (https://www.kernel.org/doc/Documentation/vm/overcommit-accounting), you've noted that `vm.overcommit_ratio` specifies the percentage of physical RAM that gets included in computing the commit limit when in "mode" 2. The default value for that is 50%, meaning that Linux will cap allocations at swap size plus 50% of physical RAM.

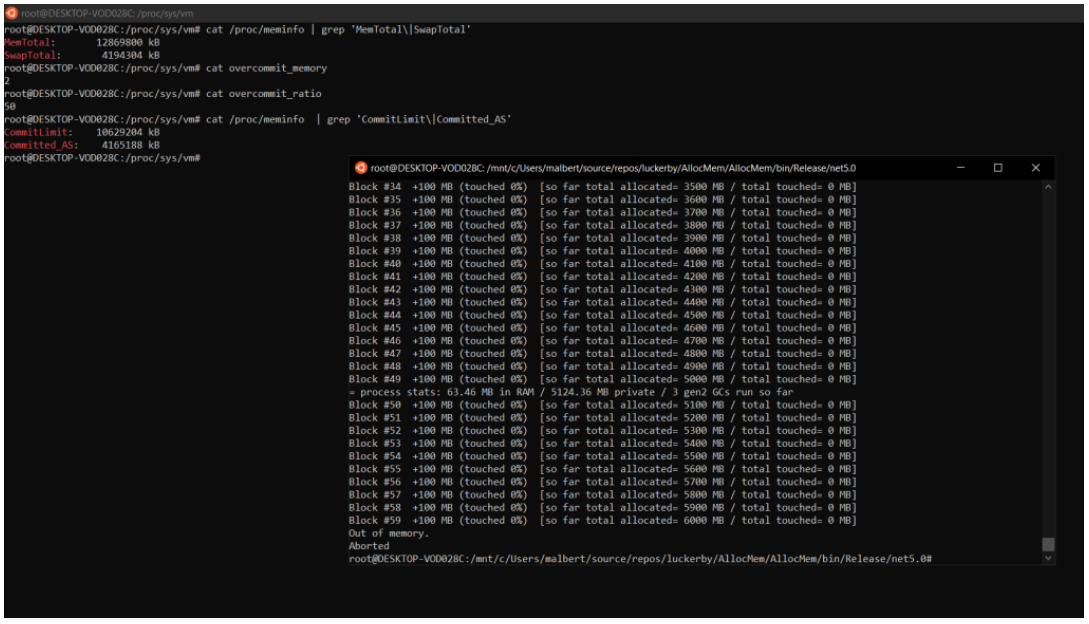
Let's test this "don't overcommit" mode by setting `vm.overcommit_memory` to 2 on the same Linux system. Here's the configuration showing the same amount of physical RAM (~12 GB) and swap (4 GB) and our new overcommit settings.



(https://luckerbyhome.files.wordpress.com/2021/11/testnoovercommit_params.png)

Figure 2 – "dont' overcommit" mode

The commit limit value shown is computed as 50% x ~12 GB + 4 GB which gives a little over 10 GB. Of course we don't expect to be able to allocate using our test tool all that much, since the kernel and other programs already running did some allocations already. How much that's worth is seen in the `Committed_AS` value which is close to 4 GB. As such, we expect to be able to allocate around 6 GB before being denied extra memory:

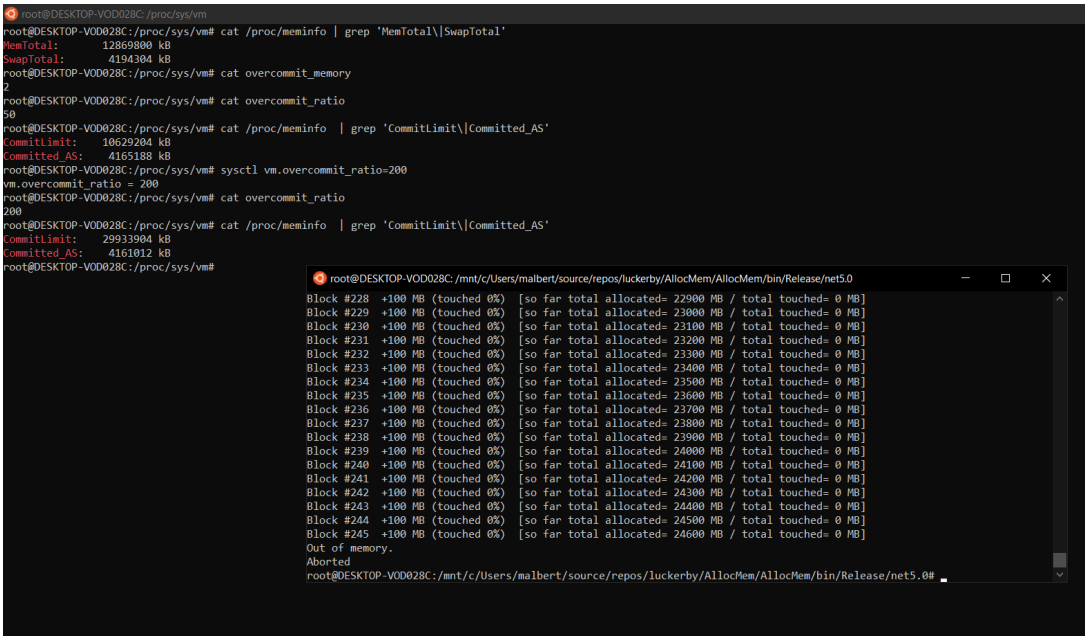


(https://luckerbyhome.files.wordpress.com/2021/11/testnoovercommit_outcome1.png)

Figure 3 – Memory allocations failing faster under the new mode

It checks out.

Note that there's no condition that the percentage in `vm.overcommit_ratio` be lower than 100% – e.g. set that to 200% and we're back in overcommit land, despite having `overcommit_memory` set to a value that is documented to disable overcommit. The test, after modifying the ratio from 50% to 200%:



(https://luckerbyhome.files.wordpress.com/2021/11/testnoovercommit_ratio_over_one.png)

Figure 4 – changing the "don't overcommit" mode parameters

So to summarize: Linux can be made to work similar to Windows from a memory management perspective in terms of how much to allocate – set the overcommit mode to 2 and a ratio value of 100% to essentially specify a commit limit of physical RAM + swap. But that's not how Linux will work by default, whereby it'll allow memory allocations without backing them up with actual memory storage – "over-commit". Should all the processes play nice and don't actually try to use a lot of their (successfully) allocated memory then all is good; when that's no longer the case the game is up and applications start being denied access to what was supposed to be their rightful own memory (but haven't yet touched up to that point ever since they allocated it).

But why have overcommit in the first place? At least 2 reasons: 1) accommodate those apps that allocate lots of memory, don't quite use all of it, but break if they can't get their memory chunks allocated in the first place (apparently there's quite a few of them under Linux), 2) when forking (which is the only way all user processes on a Linux system start) and the respective process has already allocated large quantities of memory, which in turn involves copying the virtual address space of the "source" process and in turn risking to increase the overall commit size for the system above the commit limit (details here https://stackoverflow.com/a/4597931/5853218) (https://stackoverflow.com/a/4597931/5853218)) regardless if the new process won't be actually further *using* any new memory on its own; if overcommit is enabled and set right then there's no risk of running into the commit limit as that's simply high enough (or it's not taken into account).

Is overcommit a good thing? I can see the arguments pro and against, but I'm probably biased in forming an opinion since the Windows model I've learned first doesn't allow the mechanism in the first place. However, there are heated debates around it: https://lwn.net/Articles/104179/ (https://lwn.net/Articles/104179/) .

OOM killer

Another important topic that we need to address is the OOM killer. What is the OOM killer? It's a Linux component whose job is to watch when the memory the system has is critically low and to take drastic measures to get out of that state when it happens. How does it free memory? It simply kills one process based on specific metrics and continues to do so until it determines the system is no longer critically low on memory. There's clear (and concise!) documentation here https://www.kernel.org/doc/gorman/html/understand/understand016.html (https://www.kernel.org/doc/gorman/html/understand/understand016.html) on how the OOM killer operates.

Let's invoke the OOM killer. How to do this? Quite simple, we have to make it so the actual free memory on our target Linux system comes as close to 0 as possible. This in turn causes the OS to invoke the OOM killer in an attempt to free some memory and keep things stable. Therefore our task is to exhaust what the OS can use to back up the allocated memory: physical RAM and swap.

First, we'll put the system in the "always overcommit" mode. This way we make sure there's no commit limit we're hitting first and thus rob us of the chance to see the OOM killer in action. Next, we'll use the same tool as before to allocate memory, but this time we'll touch a significant portion of it, just to make sure we're cutting down on the available memory. 50% was chosen, so for each block of 100 MB we allocate we'll be writing data to 50 MB of those. Let's see this in action below, and keep an eye on the memory and swap usage in htop:

Movie 1 – OOM killer is invoked

The process is writing to half of all the memory is allocating. That quantity eventually outgrows both physical RAM and the swap, so the system simply doesn't have anywhere else to put the data that the process asks of it. Without enough memory to operate, the kernel invokes the OOM killer. The moment when the OOM killer shows up can't be missed – the allocator tool's console receives a "*Killed*" message, while the kernel log seen to the left is very verbose as to the decisions and actions the OOM killer performs.

The key is determining when the system is truly out of memory. Will the OOM killer be invoked back in the scenario in figure 1? No, not very soon at least, as the actual memory that the system can *use* is still high. We did allocate 1 TB of memory – which under the "always overcommit" model simply means that every request for memory is granted – but we didn't really *use* that memory, so the OS never bothered to actually build the memory pages in the first place. As a result, there's no physical RAM nor swap space occupied by that 1-TB-and-going process*.

Remember that overcommit allows processes to allocate memory to their hearts' contempt (in various degrees, with the "always overcommit" mode being at the top), but when the time comes to make use of that memory, they're all fighting for the actual limited memory resources (physical RAM and swap) that the OS has.

Even with overcommit off – so having a hard commit limit equal to the sum of physical memory plus swap – the OOM killer will still be invoked when the system is running low on memory. Remember its job is to kill processes when the memory is running low, it doesn't particularly care whether overcommit is on or off. So yes, the movie seen above will unfold in exactly the same way even if overcommit is off but the commit

limit is set such as the system can run into a low memory issue (e.g. `vm.overcommit_memory=2` and `vm.overcommit_ratio=100`) and the blocks allocated would have all their memory touched (e.g. a 100% touch rate). Let's double-check:

Movie 2 – OOM killer is invoked even with overcommit off

You might find references to variables that explicitly disable the OOM killer – as opposed to simply disabling overcommit – such as `vm.oom-kill`. As of now (Nov 2021) unfortunately, this doesn't seem to exist anymore (you'll get an error trying to run `sysctl` to configure it, at least on recent versions of Ubuntu) and neither in the documentation. Then, you might rightly ask, how do you go about disabling the OOM killer altogether? It doesn't appear to be a way to do so directly, although one can do it indirectly, by making sure that *the conditions* that trigger the OOM killer in the first place are never satisfied.

We've seen that capping the commit limit to RAM size plus swap can still get the OOM killer to run, as essentially there's no failsafe buffer; but you can go further and decrease the commit limit, thus giving breathing space when all the memory that was allocated actually gets used. How low should you push the commit limit? Probably not very low, as then you'll be having plenty of buffer – in the sense of large swaths of RAM – but then again you'll never get to use all that memory that you're essentially marking off-limits.

Here's an extreme version of this below, when the maximum commit limit is set at the size of the swap only. Note how the allocator tool can only get to use an amount of RAM equal to the swap size, with the system never getting to any of the RAM above 4 GB. What stops the tool? The OS not handing out any more memory blocks since the (relatively small) commit limit has been hit:

Movie 3 – OOM killer no longer invoked, but RAM being wasted

But in general, if you're running into an issue where the OOM killer kicks in unexpectedly, perhaps it's better to address the root cause of the OOM problem directly, as opposed to disabling the OOM killer, just as this post https://superuser.com/a/1150229/1156461 (https://superuser.com/a/1150229/1156461) very well discusses. In other words, it doesn't make sense complaining there's an oil slick on the factory floor each morning, and try to find out why the janitor isn't swiping the floor clean – it'll be best to fix the engine whose casing has become loose and drips the said oil, being in danger of burning out every day now.

But how would a system behave, assuming it's both low on memory and you somehow got the OOM killer not to fire? As it turns out, there's a simple way of getting into that state: start the Linux system normally (eg `vm.overcommit_memory=0`), consume some good chunk of RAM (for example with the tool used throughout this blog post) but don't go extreme – something like 70% would do, then switch `vm.overcommit_memory` to 2 and use a low enough `vm.overcommitratio` value – the default 50% will do. This will essentially bring the system into a state where what's allocated (`Committed_AS` in `/proc/meminfo`) is larger than the commit limit (`CommitLimit` in `/proc/meminfo`). Try running any command and you should get:

```
root@DESKTOP-VOD028C:/var/log# tail -f kern.log
-bash: fork: Cannot allocate memory
```

Remember that `fork` needs to duplicate the process address space – "*the entire virtual address space of the parent is replicated in the child*" here https://man7.org/linux/man-pages/man2/fork.2.html (https://man7.org/linux/man-pages/man2/fork.2.html), and further "*under Linux, fork() is implemented using copy-on-write pages, so the only penalty that it incurs is the time and memory required to duplicate the parent's page tables*" – which means no actual new memory is consumed initially but the new process will contribute towards the commit limit with a quantity equal to what of the process that spawned it. But there's no way to move forward, as in our twisted scenario we've essentially gone past the commit limit instantly, and now Linux is denying any new memory allocation as the commit size has suddenly become way too high. On the other side the OOM killer doesn't come into the picture either – and why would it, as there's still some good portion of memory (RAM and swap) free?

cgroups

Please note that this section is not intended as an introduction to cgroups. There are excellent articles that already cover this – such as the man page for cgroups (https://man7.org/linux/man-pages/man7/cgroups.7.html). For containers, have a look at this nicely written post (https://developers.redhat.com/blog/2018/02/22/container-terminology-practical-introduction) by Red Hat. What the paragraphs below try to do is get through those cgroups notions we'll need later on to understand how certain metrics are extracted and explain the inner workings of Kubernetes out-of-memory situations.

What are cgroups and why do we care about them? cgroups are a mechanism used on Linux for limiting and accounting resources. *Limiting* as in preventing more resources to be used by a process than what it's allocated to it and *accounting* as in counting how much a process – along with its children – is using from a resource type at a particular point in time.

Why are cgroups important? Because the whole notion of containers is built on top of it. The containers' limits are enforced at the OS level via cgroups, and information about memory usage for containers is obtained from the cgroups.

An important aspect to keep in mind is that cgroups don't limit what a process can "see" in terms of resources, but what it can use. This wonderful article Why top and free inside containers don't show the correct container memory (https://ops.tips/blog/why-top-inside-container-wrong-memory/) explains why looking at the free memory from within a container doesn't report it against its set limit, but instead returns the underlying OS available memory (bonus points: the article also talks about how things work under the hood when processes running inside containers try to allocate memory, and how to trace those operations). Sort of like in a museum, where you're allowed to look freely at the art shown, but you're not permitted to get too close: you see all there is to be seen, but you're not allowed to go past certain limits.

Let's ask a rather bizarre question: does the root cgroup show data for all the processes, or only for containers? Containers themselves are backed by processes (have a look at the referenced Red Hat article above for an intro to containers) so the question could be rephrased as "does the root cgroup show data for all the processes, or only for those processes that are part of containers?". Let's try to answer this.

As per https://man7.org/linux/man-pages/man7/cgroups.7.html (https://man7.org/linux/man-pages/man7/cgroups.7.html), "*a cgroup filesystem initially contains a single root cgroup, '/'*". If you check the processes under the root cgroup (for any cgroup controller, such as memory) you'll most likely find out that they aren't the same as the ones listed with *ps aux* (which returns all the processes running on that host,

using a root shell). It turns out that a process can only belong to a specific cgroup, and the root cgroup is not special. So if a process is part of a cgroup somewhere deep down in the hierarchy of a cgroup controller, then that process won't be in the root cgroup for the respective controller. It's easy to test that all the processes(/threads) are accounted for inside the hierarchy by comparing the output of `ps aux | wc -l` with `find /sys/fs/cgroup/memory -name cgroup.procs -exec cat '{}' \; | wc -l` which will give the same value (if executed close enough one after another of course).

At this point, you'll be tempted to think that the answer to the question is "no", since after all the list of processes inside the root cgroup is just a subset of those running on the machine. This is where cgroups hierarchical accounting comes into the picture (section 6 of the cgroups documentation https://www.kernel.org/doc/Documentation/cgroup-v1/memory.txt (https://www.kernel.org/doc/Documentation/cgroup-v1/memory.txt)). It turns out that systems that do have hierarchical accounting enabled – such as the Azure Kubernetes Service (AKS) nodes, which we're going to use throughout this article – will "bubble up" statistics from the child cgroups towards the root, essentially resulting in the root cgroup reporting statistics for all the usage on that node. But what if this hierarchical accounting feature is enabled only for some cgroups? Wouldn't this break the whole "bubbling up" of the stats? Turns out that once enabled (which is easily verifiable by looking inside the `memory.use_hierarchy` file inside the root cgroup and seeing that it has 1) the value can't be easily changed: see NOTE1 in the same section that reads "*enabling/disabling will fail if either the cgroup already has other cgroups created below it, or if the parent cgroup has use_hierarchy enabled*" (if one truly wants to verify this then simply run `find /sys/fs/cgroup/memory -name memory.use_hierarchy -exec cat '{}' \;` and a long list of 1 should be produced) therefore the root cgroup memory values do refer to all the processes). As such, the answer is that the root cgroup does contain statistics for all the processes running on that machine, as long as hierarchical accounting is enabled.

But why does this matter? As we'll see later on, the node memory usage at the node level is computed by Kubernetes from the memory root cgroup's statistics. And as we've seen above, the memory root cgroup can be safely used to get memory usage information about all the processes that run on the respective node, including the OS and system components.

cgroups and Kubernetes

Let's talk about a few things that involve cgroups and Kubernetes. First, is there exactly one cgroup per container created in Kubernetes? There is just one, indeed. Have a look at https://stackoverflow.com/questions/62716970/are-the-container-in-a-kubernetes-pod-part-of-same-cgroup (https://stackoverflow.com/questions/62716970/are-the-container-in-a-kubernetes-pod-part-of-same-cgroup) for an explanation complete with an example.

Secondly, there is a notion that will creep up later on in the article, particularly in the various metric components endpoints' output: the pause container. This is nicely explained in this article https://www.ianlewis.org/en/almighty-pause-container (https://www.ianlewis.org/en/almighty-pause-container). As one pause container will be present in every single Kubernetes pods, it's worth taking the time to understand it, so that later on we'll know the reason for always seeing metrics for one additional container per pod (whose memory usage values will be extremely low).

Another thing of interest is to be able to navigate to a container's cgroup inside the memory controller hierarchy to inspect statistics. For that you need to extract details about those containers. There are several ways to go about this. One – should you use a CRI-compatible runtime is to use `crictl` as described here

https://kubernetes.io/docs/concepts/scheduling-eviction/pod-overhead/#verify-pod-cgroup-limits (https://kubernetes.io/docs/concepts/scheduling-eviction/pod-overhead/#verify-pod-cgroup-limits). Another one – should you use containerd as the container runtime (like AKS does for a while) – is to list the containers using *ctr –namespace k8s.io containers list* (note that containerd has namespaces itself as described here https://github.com/containerd/containerd/issues/1815#issuecomment-347389634 (https://github.com/containerd/containerd/issues/1815#issuecomment-347389634)). Yet another way to obtain the path to a container inside a pod is to just use cAdvisor's output, as we'll see further on in the article.

cgroups v2

cgroups v2 support in Kubernetes has been planned for a while, and you can find relevant information here (https://github.com/kubernetes/enhancements/issues/2254). As of now (Feb 2022) though – that work is still ongoing – although most of the components that Kubernetes relies on have been updated (https://github.com/giuseppe/enhancements/blob/5b4d3d5ec07b8e2ee7d231c6f99c09b0da04a48a/keps/sig-node/20191118-cgroups-v2.md#current-cgroups-usage-and-the-equivalent-in-cgroups-v2).

How can you check if a Linux OS is using cgroups v2 or not? Either via this slightly more bullet-proof method (https://unix.stackexchange.com/a/617824/470786) or the simpler `stat -f --format '%T' /sys/fs/cgroup` as seen in the text of the actual proposal to enable cgroups v2 in Kubernetes here (https://github.com/giuseppe/enhancements/blob/5b4d3d5ec07b8e2ee7d231c6f99c09b0da04a48a/keps/sig-node/20191118-cgroups-v2.md#proposal).

Even though the Kubernetes official documentation https://kubernetes.io/docs/setup/production-environment/container-runtimes/#cgroup-v2 (https://kubernetes.io/docs/setup/production-environment/container-runtimes/#cgroup-v2) has been updated to include cgroups v2 and even how to enable it on the nodes, none of the major cloud providers are offering it at this time (Feb 2022) to my knowledge (but some folks are asking for it, seen here (https://github.com/awslabs/amazon-eks-ami/issues/824) with Amazon Elastic Kubernetes Service (EKS)).

Now there are certain parts of this article – specifically the low-level discussions around the cgroup pseudo-filesystem and some of the code analysis that make the assumption that cgroups v1 are used on the Kubernetes nodes. The section that details how the OOM killer acts on cgroups when the processes inside exceed the limit is also based on the assumption that cgroups v1 nodes are employed. Why is this important? Because cgroups v2 fix some of the shortcomings of v1, for example the fact that the OOM killer doesn't care if it only kills a single process inside a cgroup and leaves the corresponding container in a broken state (see this presentation (https://www.youtube.com/watch?v=u8h0e84HxcE) at 14:45 and 17:55), so parts of that article will no longer hold when cgroups v2 becomes the norm.

For the reason stated above, we'll assume throughout the article that cgroups v1 is used unless specified otherwise.

Cgroups and the OOM killer

We've seen previously how the OOM killer will step in and act when it sees the OS in a low memory situation. With the introduction of cgroups, the OOM killer has been updated to work with them as well, as described in Teaching the OOM killer about control groups (https://lwn.net/Articles/761118/).

But if the trigger back in the OOM killer section was a system critically low on memory, what is it for cgroups? For cgroups – and specifically for the ones we're discussing in the context of Kubernetes where swap is (as of now) disabled – what causes the OOM killer to act is when the cgroup's memory usage goes above its configured limit.

Does the OOM killer start wreaking havoc inside the cgroup as soon as its aggregated memory usage crosses the defined limit even by the slightest amount? No. We know this from the documentation https://www.kernel.org/doc/Documentation/cgroup-v1/memory.txt (https://www.kernel.org/doc/Documentation/cgroup-v1/memory.txt) (section 2.5): "*when a cgroup goes over its limit, we first try to reclaim memory from the cgroup so as to make space for the new pages that the cgroup has touched. If the reclaim is unsuccessful, an OOM routine is invoked to select and kill the bulkiest task in the cgroup. […] The reclaim algorithm has not been modified for cgroups, except that pages that are selected for reclaiming come from the per-cgroup LRU list.*" The "bulkiest task" actually refers to the process eating up the most memory inside the respective cgroup (see "*Tasks (threads) versus processes*" section in the cgroups man page (https://man7.org/linux/man-pages/man7/cgroups.7.html) which tells that tasks refer to threads; also remember that in Linux threads are just regular processes sharing certain resources).

So now we know when the OOM killer decides to act against a cgroup, and also what its actions are. Let's see this unfolding.

OOM Scenario #1: Container is OOMKilled when it exceeds its limit

Movie 4 – OOM killer invoked in Kubernetes

You see a Kubernetes pod started that runs one instance of the memory leak tool (https://mihai-albert.com/2022/02/13/out-of-memory-oom-in-kubernetes-part-1-intro-and-topics-discussed/#memory-leak-tool). The pod's manifest specifies a limit of 1 GiB for the container running the app. The leak memory tool allocates (plus touches) memory in blocks of 100 MiB until the OOM killer steps in as the container runs into the 1 GiB memory limit. The top left window is the raw output from the leak tool as it allocates, the right window tails the status of the pod while the window at the bottom is tailing kernel messages. Notice that the OOM killer is quite verbose in regards to both the potential victims it analyzes as well as the process that it decides to eventually kill.

So far so good. Things are simple: the process that consumed the most, in our case the leak memory tool ran using the .NET host (dotnet), is killed when the cgroup is using too much memory.

Unexpected twists

But let's take things further and consider a slightly different example. A container is started that allocates a quantity of memory (900 MB) using our memory leak tool (process ID 15976 below). Once it successfully completes allocating the memory, `kubectl exec` is used to connect to the container and start `bash`. Inside the shell, a new instance of the memory leak tool is launched (process ID 14300 below) that starts allocating memory itself.

You can see the outcome below. Keep in mind that the PIDs are as seen at the node level, where this log was captured on. As such the process that the container starts doesn't have a reported PID of 1, but instead the actual PID on the machine. The `rss` data is in pages, so you need to multiply by 4 KB to get the size in bytes. Several unimportant lines emitted by the OOM killer have been omitted:

```
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895293] dotnet invoke
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895296] CPU: 0 PID: 1
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895297] Hardware name
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895298] Call Trace:
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895306]  dump_stack+0
.....
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895351] memory: usage
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895352] memory+swap:
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895353] kmem: usage 5
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895353] Memory cgroup
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895397] anon 1068539
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895397] file 0
.....
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895398] Tasks state
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895398] [  pid  ]   u
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895401] [  15885] 65
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895403] [  15976]
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895404] [  16234]
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895406] [  14300]
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895407] oom-kill:cons
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.895437] Memory cgroup
Jan 16 21:33:51 aks-agentpool-20086390-vmss00003K kernel: [ 8334.906653] oom_reaper: r
```

Note that the process that gets killed isn't the process the container originally started (pid 15976) despite the fact that it's using 4 times the memory consumed by the one we subsequently ran. Instead it's the latter that gets killed. At this point you might rightly conclude that it's something to do with the process doing the allocation that bumps its appeal to the OOM killer as the whole pod's cgroup goes over the limit. And it would only seem fair – why punish a process that wasn't doing anything when there's another one that was actively allocating and eventually pushed memory usage over the limit? Besides, the logs do point to which process actually triggered the OOM killer (2nd line in the output above).

Let's test that and see what happens when the roles are reversed, and the manually-run leak tool (now process id 31082) having earlier just finished allocating a small amount of memory (a total of 60 MB in blocks of 10 MB every 4 seconds), while the "main" container process (now process id 30548) is still allocating memory (blocks of 50 MB every 3 seconds) until exhaustion. Who will be killed out of the 2 when the 1 GiB limit is crossed by the whole group?

```
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076861] dotnet invoke
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076865] CPU: 1 PID: 3
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076865] Hardware name
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076866] Call Trace:
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076874]  dump_stack+0
....
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076928] memory: usage
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076929] memory+swap:
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076930] kmem: usage 5
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076930] Memory cgroup
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076941] anon 1068421
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076941] file 0
....
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076941] Tasks state
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076942] [  pid  ]   u
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076944] [  30373] 65
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076946] [  30548]
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076947] [  30617]
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076949] [  31082]
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076951] oom-kill:cons
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.076993] Memory cgroup
Jan 17 21:51:19 aks-agentpool-20086390-vmss00003M kernel: [ 1856.089185] oom_reaper: r
```

And yet another surprise: it's the manually started process that allocated the least, in the smaller blocks with the largest pause between them that gets killed. Even more, the process that triggered the OOM killer in the first place is the one consuming the most memory in the cgroup. And you might rightly say "but wait, the process started initially by the container (pid 30548 in our latest example) – is special in its own way, and the kernel probably protects it somehow from being killed, as long as it's not the last one in its cgroup". That could be so, and the fact is I simply don't know why we get this outcome. What I do know is that there is additional code aside just selecting the "bulkiest task" as the documentation tells, for example here https://github.com/torvalds/linux/blob/v5.4/mm/oom_kill.c#L334-L340 (https://github.com/torvalds/linux/blob/v5.4/mm/oom_kill.c#L334-L340), that can complicate matters. The OOM killer is still a kernel component that receives updates from time to time, and it's quite possible that not everything is reflected in the documentation presented earlier.

Despite making things slightly confusing with these last 2 examples, there is probably a takeaway from this: that one shouldn't rely on expectations in regards to which process the OOM killer will terminate inside a cgroup, beyond the fact that there will certainly be a victim once the cgroup limit is breached.

Looking at the kernel logs you might have spotted that the tasks (processes) reported in the kernel logs when the OOM killer selects a victim refer to the pod's cgroup. But the fact is there is one cgroup created per container. If you `kubectl exec` into a container to run `bash`, this new process will be made a member of the existing cgroup for the container. Any new process you start from that shell will also be a member of the same cgroup. So why do we get the statistics for the pod cgroup as opposed to the ones for the container cgroup? It's the latter that contains the leak memory tool instances. If you look carefully at the message right after the tasks state, you can see the cgroup that triggered the OOM, and it's the pod one. So we get the statistics for the pod cgroup because it as a whole went over the limit (the pause container (https://www.ianlewis.org/en/almighty-pause-container) doesn't have a limit set, and the leak memory container cgroup has the same limit as the one set for the cgroup created for the pod itself).

Kubernetes and the OOM killer

Following up on the previous sections, it's important to note that the OOM killer deciding to terminate processes inside cgroups is something that "happens" to Kubernetes containers, as the OOM killer is a Linux kernel component. It's not that Kubernetes decides to invoke the OOM killer, as it has no such control over the kernel.

Given that the OOM killer's victim is swiftly terminated, this has a powerful implication: there's no time for the targeted process to do any sort of cleanup or graceful shutdown. It never gets a chance to do anything as the kernel never gives it the opportunity to ever run again. And this (rightly) tends to upset folks running production code in Kubernetes, as you don't want any of your service's pods to just disappear all of the sudden, particularly if it's in the middle of writing something remotely (e.g. to a database). You can find such a discussion here Make OOM not be a SIGKILL (https://github.com/kubernetes/kubernetes/issues/40157#issuecomment-867548699).

Another important aspect is that the OOM killer doesn't understand the notion of a Kubernetes pod. It actually doesn't even know what containers are. Containers are constructs built on top of cgroups and namespaces, but that's out of the grasp of the OOM killer. All it knows is that when a cgroup goes over its memory limit it has to kill at least one task (or process) inside that cgroup. And this is a problem for Kubernetes as it deals with containers at its lowest level. So having processes inside a container disappear – particularly any that's not the main container process – puts Kubernetes in a tough spot: the container has just lost functionality, but not enough to terminate the container completely (the process it started initially with pid 1 is very much alive), so in essence it's now crippled. This exact problem is in this old thread here Container with multiple processes not terminated when OOM (https://github.com/kubernetes/kubernetes/issues/50632) that tells that child processes in a container are killed and the whole pod isn't marked as OOM. As one of the Kubernetes committers says "*Containers are marked as OOM killed only when the init pid gets killed by the kernel OOM killer. There are apps that can tolerate OOM kills of non init processes and so we chose to not track non-init process OOM kills[…]I'd say this is Working as Intended*".

And we've seen this exact scenario in the last 2 examples of the previous section Cgroups and the OOM killer, where a container kept running despite one of the processes inside having been killed by the OOM killer.

So if the OOM killer operates on its own and can kill processes that can potentially result in containers being stopped, how does Kubernetes live with this? It can't possibly manage the pods and their containers successfully if the latter suddenly disappear. As we'll see further down this article (Is Kubelet killing containers due to OOM? (https://mihai-albert.com/2022/02/13/out-of-memory-oom-in-kubernetes-part-4-pod-evictions-oom-scenarios-and-flows-leading-to-them/#is-kubelet-killing-containers-due-to-oom)) Kubernetes is very much aware when OOM killer events are generated, and as such can reconcile its own state with what goes on inside each node.

The very name of the OOM killer makes one think that once it acts, it stops its victim forever. But that's not the case when dealing with Kubernetes, as by default if a container terminates – regardless if it successfully exited or not – it's restarted. And in this case the outcome you'll see is OOMKilled containers being restarted endlessly (albeit with an exponential back-off delay (https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#restart-policy)).

One last thing before we finish this section: as we've noticed, the OOM killer is rather unforgiving once the cgroup's memory limit is crossed (and if nothing can be reclaimed, of course). This naturally leads one to wonder if there's no sort of soft limit in place, as to avoid the sudden termination of containers and give some sort of early notice that memory is filling up. Even though cgroups have support for soft limits (see section 7 here https://www.kernel.org/doc/Documentation/cgroup-v1/memory.txt (https://www.kernel.org/doc/Documentation/cgroup-v1/memory.txt)), Kubernetes doesn't make use of them as of now (note that this is about cgroups' soft limits, with a relevant document here (https://gist.github.com/mcastelino/b8ce9a70b00ee56036dadd70ded53e9f#memory-resource-management), not about soft eviction thresholds in Kubernetes described here (https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/#soft-eviction-thresholds)).

Runtime implications around OOM

There's a high chance that the applications you'll be running inside your containers will be written in a language that's using a runtime, such as .NET or Java. But why would this matter from the standpoint of memory allocation?

There are languages without a runtime – such as C, which only has a runtime library, but no underlying "environment" runtime – where you're free to allocate memory to your heart's content. So normally (normally in the sense without using things that can limit the maximum memory, such as cgroups) you'd be able to use all RAM (and swap) if you allocate enough, just as long as the OS allows those allocations.

There are other languages – like Java or those based on the .NET platform (C#) – that rely on runtimes, where the approach is different whereby the application doesn't go by itself and allocate virtual memory, but instead it's the runtime that takes care of it, along with additional things like mapping the objects allocated to a generational model, invoking the garbage collector etc. The runtime will act as a middleman between the app and the OS as it needs to keep track of which objects are used, if the memory around them can be reclaimed by the GC since they're no longer in use, etc.

But it's not the intricacies of how a runtime allocates memory (https://mihai-albert.com/2019/12/15/boxing-performance-in-c-analysis-and-benchmark/#too-much-of-a-surprise) that we care about right now. Instead, it's the fact that runtimes can have settings in place that limit the maximum memory an application can allocate. So we have to add one potential culprit when conducting an investigation as to why an application cannot allocate memory: it's not enough to ensure that the OS of the underlying Kubernetes node has the memory requested and that no cgroup limits the memory for our application in its containers, but the runtime must allow the allocation as well.

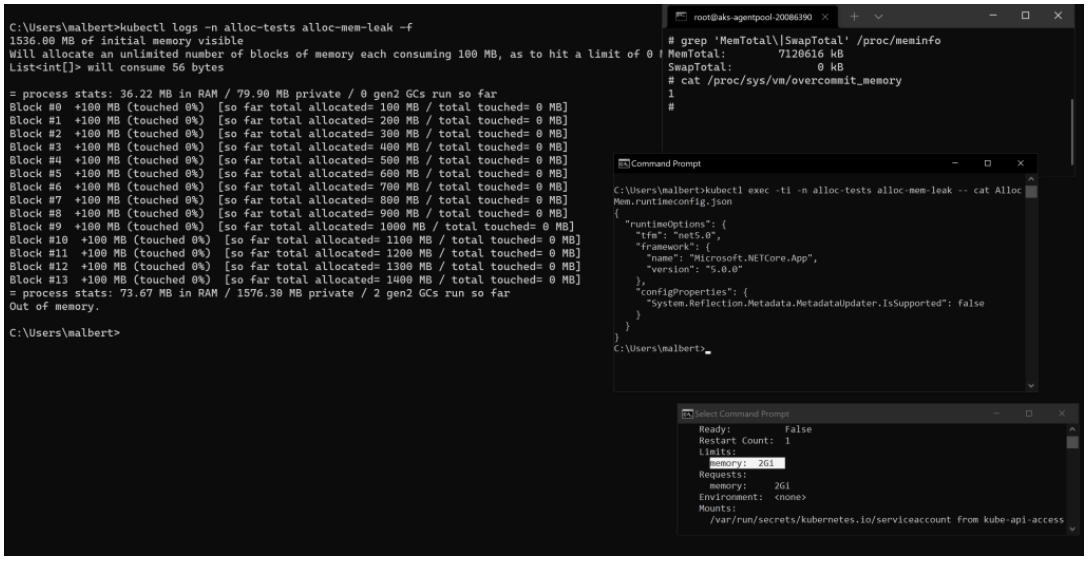
A follow-up question would be why would we care about the fact that the runtime can set a limit on the maximum memory to be used by an application, if we're not actually setting any limits ourselves? The answer is that sometimes the defaults prevent all the available memory to be used, as we'll see next.

.NET

.NET has settings that can limit how much heap is used. What is the heap? Simply put, the place where most of a regular application's objects will end up (for a more elaborate discussion, go here (https://mihai-albert.com/2019/08/04/boxing-performance-in-c-sharp-the-basics/#which-goes-where)). Where is this heap located? Simplifying again – it takes space in memory. So if I were to allocate and use a number of `int` arrays that take overall 1 GB, that space will be used from the heap and in turn, there must be 1 GB of memory to accommodate that data. I do want to stress again that this is an oversimplification – concepts like virtual memory, overhead in the form of page tables, lazy evaluation when allocating memory and other details are needed to give the whole picture – but for what we're after in this blogpost our simplified model of the world will do.

One of the settings that .NET uses to control the maximum amount of memory that can be allocated is `System.GC.HeapHardLimitPercent (https://docs.microsoft.com/en-us/dotnet/core/run-time-config/garbage-collector#heap-limit-percent)` (or one of its equivalent environment variables). The interesting part is not so much the default value of it – "*the greater of 20 MB or 75% of the memory limit on the container*" – but when it gets to kick in: "*The default value applies if: The process is running* ***inside a container that has a specified memory limit***". So you'll get a different behavior *out-of-the-box* when running your .NET code inside a "regular" application as opposed to containerizing it.

Let's test using a container that has a memory limit set to 2 GB and that's running our memory leak tool, itself written in .NET. We'll just allocate memory but not touch any of it, as our goal is to see how the runtime handles allocations. The left side of the screen below captures the output inside the container as allocations take place, while for the console windows to the right: in the first one we see the total available memory for the underlying Kubernetes cluster node and the fact that overcommit is enabled (by default in AKS), in the second the configuration for the .NET app (the memory leak tool) showing no setting enabled for the maximum heap size (nor were there any environment variables set to control it), and in the third the memory limit for the container.



(https://luckerbyhome.files.wordpress.com/2022/02/runtimestoppingallocationswhenlimitisset.png)

Figure 5 – .NET runtime stopping memory allocations when container limits are set

The memory allocated overall by the app – which includes some overhead due to the .NET runtime itself – crosses the threshold of 75% out of the 2 GB container limit, and the runtime terminates the app as it cannot provide another memory block.

How can the default limit be bypassed? Just specify a different percentage either by using the JSON setting or one of the 2 environment variables currently defined in the documentation for heap limit_percent (https://docs.microsoft.com/en-us/dotnet/core/run-time-config/garbage-collector#heap-limit). If you go with the JSON option, either specify the value after you've built the app or just use a template file as described here (https://docs.microsoft.com/en-us/dotnet/core/run-time-config/#runtimeconfigjson) so it doesn't get overwritten every time.

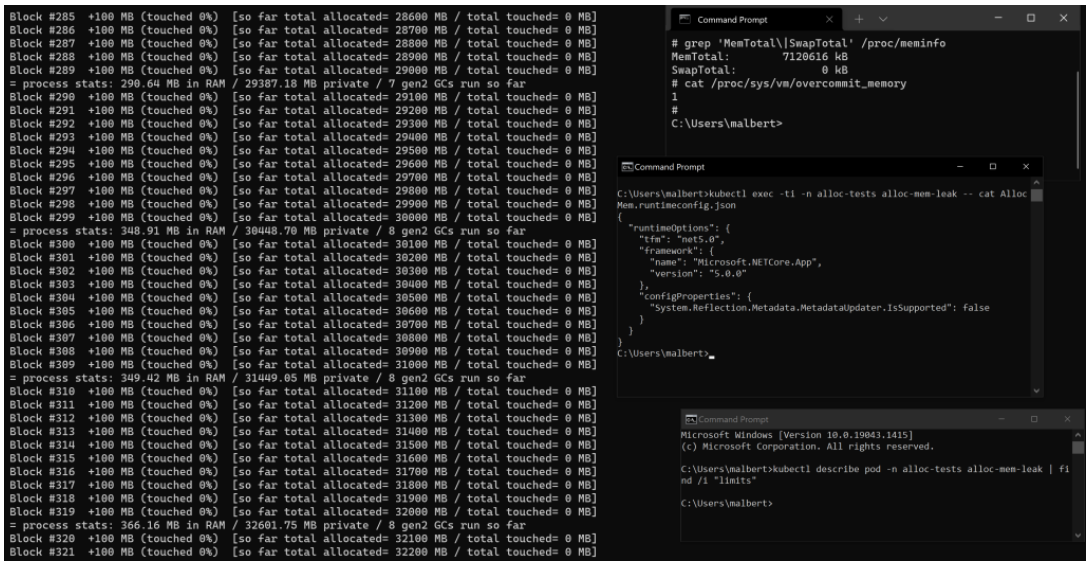
If you're looking to remove the runtime limits on memory allocations when inside a container, be aware that the percent value used can't be just anything, as there's an `if ((percent_of_mem > 0) && (percent_of_mem < 100))` inside the GC implementation here (https://github.com/dotnet/runtime/blob/1c927462696f5af7d5e667f54b1c39452cba4b2d/src/coreclr/gc/gc.cpp). But there's an alternate setting, `System.GC.HeapHardLimit` (https://docs.microsoft.com/en-us/dotnet/core/run-time-config/garbage-collector#heap-limit) that allows one to specify an absolute value, which isn't validated in any way.

Would you want to actually push the heap limit to 100%?. Probably not, as Jan Vorlicek explains in this GitHub issue (https://github.com/dotnet/runtime/issues/61076#issuecomment-956968334) I've opened that "*.NET GC uses 75% of the container memory limit as its default allocation limit. The limit was chosen to leave enough space for native parts of .NET and 3rd party libraries that .NET uses*".

An important note is that the setting refers to memory *allocated* and not necessarily *used*. Why is this important? When running inside a container with a memory limit, Linux will kill the process that takes the most inside the respective cgroup once the actual *usage* goes past the limit. But an application can allocate memory just fine but not use it, and in this case it's the runtime that will be tripped on a setting that can be far higher than the container's limit.

An example: assume the .NET runtime has a heap hard limit of 20 GB but the limit for the container where it runs is 2 GB. Also assume that the overcommit settings for the OS don't prevent allocating over 20 GB. Should the application allocate but not use any of that memory, the actual memory usage for the container won't be anywhere near 2 GB by the time the allocated memory runs into the 20 GB limit. And it will be the runtime that will cause the application to stop, not the underlying OS.

What about if the container doesn't have a limit set? Will the .NET runtime block any allocations by applying the default value for `System.GC.HeapHardLimitPercent` (75%) to the underlying's node physical memory size? After all, this is what a container "sees" in terms of maximum memory (as we've seen back in the cgroups section, containers see the whole memory size of the node). The answer is no – it's only when limits are set for the container that this mechanism kicks in. Here's the same exact test we've done before, but this time the container limit has been removed:



(https://luckerbyhome.files.wordpress.com/2022/02/runtimenotstoppingallocations.png)

Figure 6 – .NET runtime allowing allocations in a container without limits set

More than 30 GB of memory have been successfully allocated and the tool is still going. With overcommit turned on at the OS level – which essentially allows any memory allocation request -, the app not touching the allocated memory as for the OS to have to use any actual RAM, and without any limit enforced by the runtime as the setting for the heap is disabled, there's nothing preventing our app from allocating large amounts of memory.

To conclude, keep in mind that the runtime can have a say in the maximum amount of memory allocated by an application, and this should be investigated as well when one is dealing with memory allocation issues.

For completeness, it must be noted that the Go language does things slightly different: it doesn't have an actual runtime, but instead places the GC functionality inside a runtime library (details here (https://go.dev/doc/faq#runtime)). What Go is currently missing (as of Dec 2021) is a setting to specify the maximum limit for the heap. But the community has – and still is – asking for this feature, and if and when that is implemented one would have to keep it in mind when troubleshooting memory issues inside containerized apps, just like we've seen at length with the .NET runtime.

Kubernetes resource requests and limits

It's important to understand what resource requests and limits mean in Kubernetes, and now is the time to make sure, as further down in the article things like containers getting OOMed and pod evictions will make heavy use of resource requests and particularly limits.

The Resource Management for Pods and Containers (https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/) official Kubernetes article – particularly the first half of it – is great at explaining the concepts of resource requests and limits. The core idea follows: "*when you specify the resource request for containers in a Pod, the kube-scheduler uses this information to decide which node to place the Pod on. When you specify a resource limit for a container, the kubelet enforces those limits so that the running container is not allowed to use more of that resource than the limit you set*". Further down

in that article (https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/#how-pods-with-resource-limits-are-run) you'll also find a glimpse of how the limits are implemented under the hood by using cgroups.

Some good articles that go over the concepts of resource requests and limits:

- Kubernetes best practices: Resource requests and limits (https://cloud.google.com/blog/products/containers-kubernetes/kubernetes-best-practices-resource-requests-and-limits) presents the concepts of request and limit clearly and concise
- A Deep Dive into Kubernetes Metrics — Part 3 Container Resource Metrics (https://blog.freshtracks.io/a-deep-dive-into-kubernetes-metrics-part-3-container-resource-metrics-361c5ee46e66): there is a section dedicated to requests and limits that describes the notions nicely. I strongly recommend reading the whole series of articles, as it discusses other topics very well and in detail

It's here that we must talk about QoS classes for pods. To be honest, every time I encountered this concept I just waved my hand and thought "nah, I don't need this yet"; I thought I could learn faster how the components involved in OOM situations work without knowing what QoS classes mean, but eventually I just ended up wasting far more time overall than the 5 minutes it would have taken to learn the notion. Here are 2 reasons why they matter: 1) they dictate the order in which pods are evicted when memory on the node becomes scarce and 2) they decide where in the root memory cgroup `/sys/fs/cgroup/memory/` hierarchy on the Linux node will the cgroups for pods and their containers be placed (if the `--cgroups-per-qos=true` flag is supplied to the Kubelet, which is the default in current versions) which in turn makes it easy to browse to the correct directory in the pseudo filesystem and see the various details about the cgroups belonging to a specific pod.

The QoS classes for the pods are not something one manually attaches to pods. It's just Kubernetes deciding based on what requests and limits are assigned to the containers in that pod. In essence, Kubernetes goes through an algorithm – or a logical diagram if you will – looks at the requests and limits set on the containers inside a pod – and in the end assigns a corresponding QoS class to the pod. *Guaranteed* is assigned to pods that have all of their containers specify resource values that are equal to the limits ones for both CPU and memory respectively. *Burstable* is when at least one container in the pod has a CPU or memory request, but it doesn't meet the "high" criteria for *Guaranteed*. The last of the classes – *BestEffort* – is the case of a pod that doesn't have a single container specify at least one CPU or memory limit or request value. The official article is here Configure Quality of Service for Pods (https://kubernetes.io/docs/tasks/configure-pod-container/quality-service-pod).

Q&A

Q&A: Overcommit

Q: Assuming overcommit is set to disabled, is there any other parameter that controls the commit limit aside `vm.overcommit_ratio`?
A: Yes, an absolute value can be set via `vm.overcommit_kbytes`. Note than using this will disable `vm.overcommit_ratio` and make that one read 0 when read. Take a look at the documentation https://www.kernel.org/doc/Documentation/sysctl/vm.txt (https://www.kernel.org/doc/Documentation/sysctl/vm.txt) for more details.

Q: Does Kubernetes use overcommit on its Linux nodes?
A: It does, at least on AKS. As of now (Nov 2021) on AKS Linux nodes, the overcommit policy is set to blindly accepts any allocations requests without checking against the commit limit – in other words to always overcommit (details here https://www.kernel.org/doc/Documentation/vm/overcommit-accounting (https://www.kernel.org/doc/Documentation/vm/overcommit-accounting)):

```
root@aks-agentpool-20086390-vmss00000E:/proc/sys/vm# cat overcommit_memory
1
```

Q: Didn't you say in a previous post that you haven't found a way to display the committed memory size in Linux? What about `Committed_AS` in `/proc/meminfo`?
A: That gives how much has been committed system-wide, an equivalent to Process Explorer's System Information window "Commit Charge" section on Windows. But still not sure how to get that info at process level.

Q: How can the parameters controlling overcommit be made permanent?
A: Use `systcl -p`

Q&A: OOM killer

Q: Where can I find a good explanation of what goes into what log under Linux? I'm having trouble locating the info I want.
A: Start with this excellent SO thread https://askubuntu.com/questions/26237/difference-between-var-log-messages-var-log-syslog-and-var-log-kern-log (https://askubuntu.com/questions/26237/difference-between-var-log-messages-var-log-syslog-and-var-log-kern-log).

Q: In your movie showing OOM killer being invoked, did you have to start the rsyslog service on the underlying Ubuntu WSL before running `dmesg`?
A: There's no need to start the rsyslog service as the kernel messages will be shown just fine.

Q: I don't really care about the OOM killer running, but I just want it to avoid killing my process. How do I do that?
A: You're probably looking at setting `oom_score_adj` for your process. Have a look at this thread https://serverfault.com/questions/762017/how-to-get-the-linux-oom-killer-to-not-kill-my-process?rq=1 (https://serverfault.com/questions/762017/how-to-get-the-linux-oom-killer-to-not-kill-my-process?rq=1) for more info.

Q: But the process shown back in figure 1 must be using some RAM at least, right?
A: Of course – the image itself, various modules it loaded and .NET runtime internals do take some some memory.

Q&A: cgroups

Q: Where can I find articles that talk about cgroups in detail?
A: Aside the ones already mentioned in the post:

- A good article about multiple topics related to cgroups: https://engineering.linkedin.com/blog/2016/08/don_t-let-linux-control-groups-uncontrolled (https://engineering.linkedin.com/blog/2016/08/don_t-let-linux-control-groups-uncontrolled)
- A detailed list of the files present in the `memory` subsystem for a cgroup is here https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/6/html/resource_management_guide/sec-memory (https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/6/html/resource_management_guide/sec-memory)

Q&A: Cgroups and the OOM killer

Q: How can I see the limits set for the pod's containers' cgroups?
A: The article here Pod overhead – Verify Pod cgroup limits (https://kubernetes.io/docs/concepts/scheduling-eviction/pod-overhead/#verify-pod-cgroup-limits) presents a way to extract a pod's memory cgroup's path and shows how to inspect the pod's cgroup memory limit.
Kubernetes, memory

One thought on "Out-of-memory (OOM) in Kubernetes – Part 2: The OOM killer and application runtime implications"

1. **OLIVER**
   December 1, 2022 / 3:31 pm
   Thanks a lot for the set of very insightful and rigorous articles!

   Reply

Blog at WordPress.com.